

Tello Control — Technical Documentation

Gesture- and voice-controlled DJI Tello, fully local, hardware-optional

Marcelo Leindl

2026

Contents

1. Overview & motivation	1
2. System architecture	2
2.1 The backend abstraction	2
2.2 Data flow	2
3. Component deep-dives	2
3.1 Core — <code>tello_control.core</code>	2
3.2 Gesture control — <code>tello_control.gesture</code>	3
3.3 Voice control — <code>tello_control.voice</code>	3
3.4 Simulation — <code>tello_control.sim</code>	4
4. Technology stack	4
5. Setup & run	5
6. Engineering decisions & lessons	5
6.1 Explored and ruled out — lip-reading control	5
7. Status & roadmap	7

1. Overview & motivation

Tello Control lets a DJI Tello drone be flown by hand gestures and spoken commands, with everything running locally on the laptop. No cloud, and no internet while the drone is in the air. The defining idea is a *hardware-optional* architecture: the whole control stack can be built, tested and shown off without owning or risking a real drone, because one abstraction drives three interchangeable backends:

- a software mock (pure logic, instant, no Wi-Fi),
- a PyBullet physics simulation (real quadrotor dynamics in a 3D window),
- the real Tello over Wi-Fi.

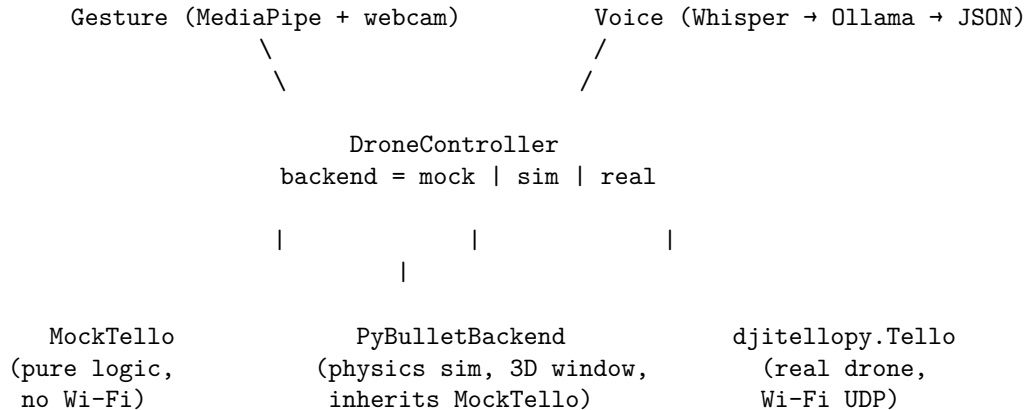
This matters because drones are unforgiving to debug. A wrong sign in a movement command, a missing safety check or a flaky gesture classifier all end the same way, with a crashed drone. Putting the mock and the simulation behind the *same* interface as the real drone lets the risky parts (gesture logic, language parsing, command validation) get hardened off-hardware first, and only then pointed at real hardware by changing one argument.

This document explains what each component does, which libraries it uses, and how they connect.

2. System architecture

2.1 The backend abstraction

Everything above the controller is backend-agnostic:



`DroneController` (`tello_control/core/controller.py`) exposes a small, stable verb set — `connect`, `takeoff`, `land`, `emergency`, `forward/back/left/right/up/down`, `rotate_cw/ccw`, `disconnect` — and forwards each call to the chosen backend. The gesture and voice modules only ever see this interface; they never import a concrete drone class. Switching from simulation to a real flight is literally one argument: `DroneController(backend="sim")` → `DroneController(backend="real")`.

The real-drone class (`djitellopy.Tello`) is imported lazily, only when `backend="real"` is requested, so the package runs fine on a machine that never connects to a drone. The same is true for the `PyBullet` backend, which lives in a separate conda environment.

2.2 Data flow

Both input modalities converge on the same controller:

- **Gesture:** webcam frame → `MediaPipe` hand landmarks → angle-based classifier → a `Gesture` enum → `debounce` → a `DroneController` method call.
- **Voice:** microphone → voice-activity detection + wake word → speech-to-text → local LLM → JSON command list → validation → a sequence of `DroneController` calls.

3. Component deep-dives

3.1 Core — `tello_control.core`

`MockTello` is a software stand-in for `djitellopy.Tello`. It mirrors the real SDK’s method names but, instead of flying, it maintains a simulated 3-D pose (`x`, `y`, `z` in cm and `yaw` in degrees), logs every command with a timestamp, and enforces the real SDK’s limits (distance 20–500 cm, angle 1–360°) by raising a `TelloException` on violations. It can print a command log and an ASCII top-down map of the flight path. This is what makes “test the whole control logic at your desk” possible.

`DroneController` is the abstraction layer described above. It also offers `position()` and `report()` helpers that are only meaningful when a pose is tracked (mock and sim), and a `tick()` method that advances the simulation physics one step (a no-op for mock/real).

The inheritance trick: the simulation backend (`PyBulletBackend`) *subclasses* `MockTello`. It therefore inherits the entire logic layer — pose tracking, logging, the map, and the SDK safety checks — for free. It overrides only the flight methods, which first call `super()` (analytic pose update + logging + checks, exactly

like the mock) and *then* drive the real physics toward that pose. The command semantics stay identical to the mock; the simulation just adds real flight dynamics on top.

3.2 Gesture control — `tello_control.gesture`

Pipeline: **webcam** → **MediaPipe** → **classifier** → **debounce** → **command (async)**.

- `detector.py` runs MediaPipe's `HandLandmarker` (Tasks API) on each webcam frame and classifies the hand. Classification works from joint angles rather than fingertip positions: a finger counts as extended when the angle at its PIP joint (MCP → PIP → TIP) is large (~straight). That holds up under hand rotation, tilt and size, which a position-based check does not. Thumb direction is read from the thumb tip relative to the knuckle line; pointing direction uses the index finger's angle to the vertical with a $\pm 30^\circ$ dead zone for "forward".
- `command_map.py` maps each `Gesture` to a controller call, with a **debounce**: a gesture must be held for `STABLE_FRAMES` (5) consecutive frames before it fires, and a `COOLDOWN_FRAMES` (10) pause then blocks repeats. This prevents jitter and accidental double-triggers. Both were halved from their original values (8 / 20) to cut the gesture-to-command latency.
- `runner.py` runs flight commands on a **worker thread** (`AsyncCommandRunner`) so the blocking flight call doesn't freeze the webcam/preview loop. A `ThreadedCtrlAdapter` makes the async runner look like a normal controller, so the mapping code stays unchanged. While a command runs, new ones are dropped (flight commands must not stack up).
- `app.py` is the main loop: it opens the webcam, draws an overlay (current gesture, a debounce progress bar, drone state), advances the sim physics each frame via `tick()`, and handles keys (`t` takeoff, `e` emergency, `q` quit).

Gesture map: fist → hover, thumb up/down → up/down, index up/left/right → forward/left/right, peace → back, open hand → land.

Continuous velocity mode (--rc, opt-in). The default path sends discrete 30 cm `move_*` commands, each of which blocks until the drone acknowledges it — roughly 200 ms of dead time per command. `velocity_map.py` offers the alternative: a *held* gesture is translated into an RC setpoint and pushed with `send_rc_control`, which is fire-and-forget. Motion begins the moment the gesture is recognised.

`VelocityBlender` ramps the output toward the target by at most `MAX_STEP` per axis per frame. That smooths acceleration and braking, and it makes the frame debounce unnecessary as a side effect: a single misclassified frame nudges the setpoint by one step and is pulled straight back, so it never becomes visible motion. A dead zone suppresses jitter near zero. `--rc --sim` raises on purpose: driving a velocity setpoint through the PyBullet PID controller is a separate piece of work.

3.3 Voice control — `tello_control.voice`

Pipeline: **mic** → **VAD + wake word** → **Whisper** → **Ollama** → **JSON** → **validation** → **execute**.

- `stt.py` records 16 kHz mono audio (`sounddevice`) and transcribes German speech with **faster-whisper** (small model, int8, CPU). The model is loaded once and reused.
- `listener.py` powers the always-listening mode. A pure-numpy **energy VAD** (RMS threshold, auto-calibrated to the room's noise floor) cuts the mic stream into speech segments. A command is only forwarded if the transcript starts with the **wake word "Drohne"**, which is then stripped — this is the safety gate in continuous mode.
- `llm_parser.py` sends the transcript to a **local LLM via Ollama** (qwen2.5:3b, JSON mode, temperature 0) with a German system prompt and few-shot examples (including tricky number words like "ein halber Meter" = 50 cm). It also has an autostart helper (`ensure_ollama`) that launches the Ollama server if needed and checks the model is pulled. The Ollama call is deliberately separated from parsing so the parse/validate logic is testable without a running server.
- `fastpath.py` short-circuits the LLM for unambiguous single-clause commands ("lande", "vor 50 cm", "dreh dich rechts"). Anchored regexes map the transcript straight onto the command schema, saving

the ~0.8 s Ollama round-trip. Anything with conjunctions, or any value outside the SDK bounds, returns `None` and falls through to `llm_parser.parse()`. Crucially the fastpath emits its result through the same `validate_list()` as the LLM path, so it is a latency shortcut, never a safety bypass.

- `commands.py` is the **safety-critical validation layer**. The LLM returns a JSON command list; every command is checked against an allow-list of actions and the SDK bounds *before anything reaches the drone*. The contract is all-or-nothing: if any command is invalid, **nothing** executes — no half-dangerous partial flight.
- `app.py` ties it together with two modes: an ENTER-to-speak mode (with a confirmation step before real flights) and the continuous wake-word mode.

3.4 Simulation — `tello_control.sim`

Runs in the separate conda environment `tello-sim` (see §5).

- `pybullet_backend.py` wraps **gym-pybullet-drones** (built on PyBullet). On `connect()` it spawns a `CtrlAviary` with a `Crazyflie` model and a `DSLIPIDControl` PID controller, opens a 3-D window, and (optionally) follows the drone with the camera. Coordinates are converted between the `MockTello` convention (cm, clockwise yaw) and the PyBullet world (m, counter-clockwise yaw). It supports a *cooperative* mode where flight commands don't block: they only set the target pose, and the physics is advanced one slice per frame via `tick()` from the main thread — needed for interactive gesture/voice loops, because PyBullet's GUI must run on the main thread.
- `control_lab.py` is a small control-engineering lab: it flies a 1-metre step in X with different proportional gains and measures overshoot and settling time, saving a plot to `results/control_lab_step.png`. The real Tello can't do this, because its SDK only exposes velocity setpoints and no motor-level access. That is exactly why it lives in the simulation.
- `demo.py` flies the cube demo in the 3-D window; `launcher.py` is a menu that starts each sim program in its own process (so OpenCV and PyAV, which both bundle ffmpeg, never collide in one process).

4. Technology stack

Layer	Library / tool	Role	Why
Real drone	djitellopy	Tello SDK over Wi-Fi (UDP)	de-facto Python Tello SDK
Logic mock	MockTello (own)	hardware-free drone	test logic at the desk
Physics sim	PyBullet / gym-pybullet-drones	real quadrotor dynamics + PID	native on Apple Silicon, no Docker; ready-made controllers
Gestures	MediaPipe	hand landmarks	fast, local, robust hand tracking
Vision I/O	OpenCV	webcam capture + overlay	standard
Speech ⇄ text	faster-whisper	local STT	accurate German STT, fully offline
Audio I/O	sounddevice	mic capture	simple numpy-native streaming

Layer	Library / tool	Role	Why
Text $\rightarrow$ command	Ollama (qwen2.5:3b)	local LLM JSON parsing	offline, small, JSON mode
Controller input	pygame	PS4 controller	manual flight option
Tests	pytest	hardware-free test suite	guards logic + safety

5. Setup & run

Two environments are used:

1. **Main** (venv, Python 3.11+): mock, real drone, gestures, voice. `pip install -e .` then `python scripts/download_model.py`.
2. **Simulation** (conda env `tello-sim`): PyBullet. Created from `environment-sim.yml`. PyBullet is installed via **conda-forge**, **not pip**, because on Apple Silicon the pip build of PyBullet fails (its bundled zlib uses old C that recent Apple clang rejects).

Full command list: see [COMMANDS.md](#).

6. Engineering decisions & lessons

- **Gazebo $\rightarrow$ PyBullet.** Gazebo on Apple Silicon meant Docker and a painful GUI; PyBullet runs natively, gives real quadrotor physics and ready-made PID controllers — a better fit for a laptop-only project.
- **conda vs pip on M1.** PyBullet’s pip build breaks on Apple clang; conda-forge ships a working binary. `gym-pybullet-drones` is installed editable with `pip install --no-deps` so it can’t pull a second PyBullet or drag in SB3 + PyTorch.
- `setuptools < 81`. `gym-pybullet-drones` still uses `pkg_resources`, removed in newer `setuptools` — pinned in the sim environment.
- **OpenCV $\rightarrow$ PyAV conflict.** Both bundle `ffmpeg`, causing duplicate-class warnings. The sim launcher therefore runs each mode in its own process.
- **Threading model for gestures.** Flight calls `block`; the webcam loop must not. A single worker thread owns the drone, the main thread owns the camera and the PyBullet GUI — no race conditions, smooth preview.
- **Validation as a safety gate.** The voice path treats the LLM as untrusted: its output is validated against an allow-list and SDK bounds, all-or-nothing, before any command reaches the drone.

6.1 Explored and ruled out — lip-reading control

A lip-driven control channel was prototyped and ultimately dropped. Two approaches were tested:

1. **Lip-shape templates (MediaPipe FaceLandmarker + nearest-neighbour).** A 30-second per-user calibration recorded one normalised 80-dim lip vector per command, matched live with debounce/cooldown — the same architecture as the gesture pipeline. It runs in real time and is fully local, but the mouth-shape vectors for distinct commands were not separable enough on a single face: short words spend most frames near the neutral (closed-mouth) shape, so matching was unreliable even with a neutral-frame filter and multi-take calibration.
2. **True lip-reading (Meta AV-HuBERT, visual speech recognition).** The full VSR pipeline was reproduced locally: webcam $\rightarrow$ `dlib` 68-point landmarks $\rightarrow$ mean-face affine alignment $\rightarrow$ 96×96 mouth ROI $\rightarrow$ centre-crop 88 $\rightarrow$ AV-HuBERT base (LRS3-433h) $\rightarrow$ beam search. Preprocessing was confirmed correct (clean landmark detection and aligned ROI sequences), and the model ran end-to-end. Accuracy, however, was unusable for control: “take off” decoded to “*land earth*”, and “please turn

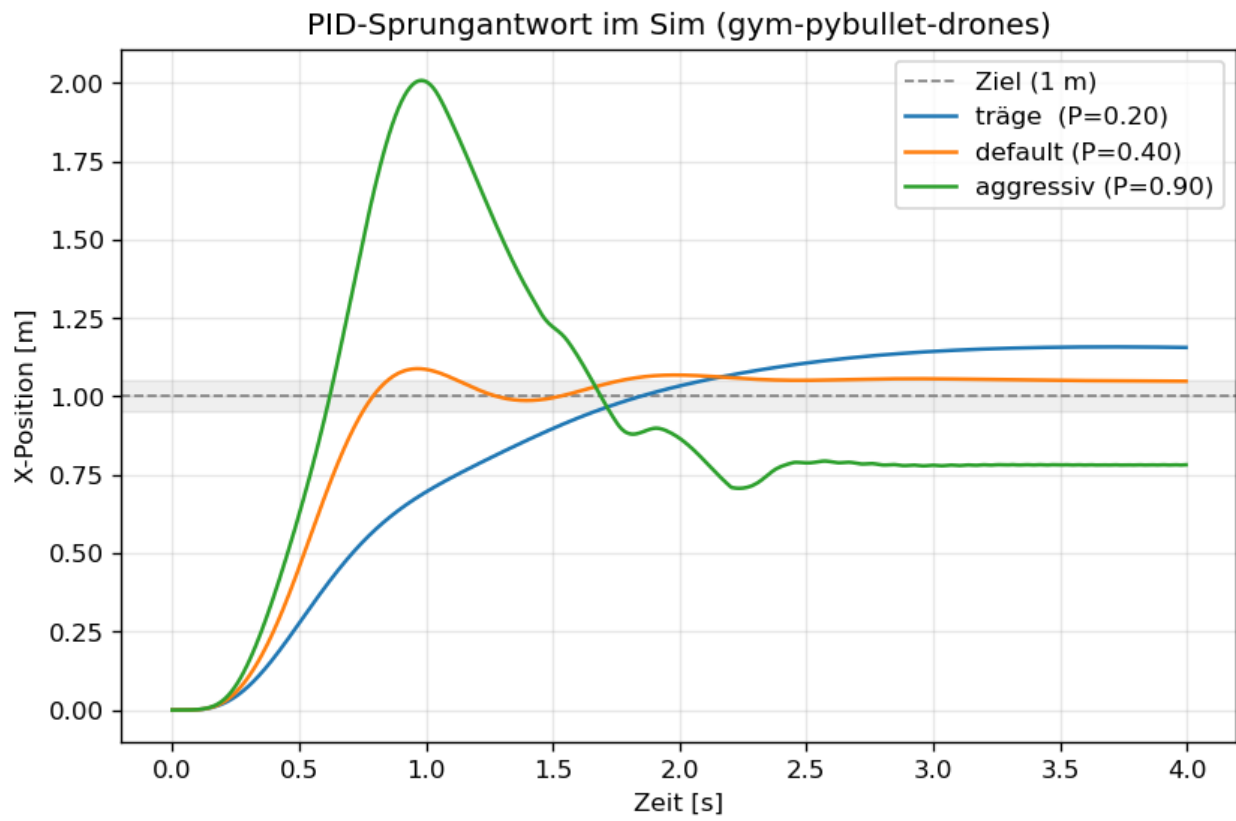


Figure 1: PID step response measured in the simulation: a gentle gain ($P=0.20$), the default ($P=0.40$), and an aggressive gain ($P=0.90$) that overshoots heavily.

off and fly up” to “*but lies a lot of guidance overly helping*” — fluent, hallucinated LRS3-style English rather than the spoken words. **Root cause:** AV-HuBERT is trained on natural connected British speech; short isolated commands are its worst case, compounded by a non-native speaker off the training distribution. The ~5 s clip-buffer + inference latency also rules it out for reactive flight.

Conclusion. Speaker-independent, low-latency lip reading is not achievable off-the-shelf today; making AV-HuBERT usable would require fine-tuning on a per-user command dataset — a separate ML project. The lip-shape prototype was removed from the codebase. Gesture and voice remain the two working control channels.

7. Status & roadmap

- **A0 Foundation** □ — controller abstraction, MockTello, first real flights.
- **A1 Gesture control** □ — MediaPipe, angle-based classifier, debounce; flown on the real drone in both discrete and `--rc` velocity mode.
- **A2 Voice control** □ — Whisper □ Ollama □ validated JSON, wake word; flown on the real drone.
- **A3 Physics sim** □ — PyBullet backend + PID control lab.
- **A4 Integration** □ — unified app, portfolio polish.